```
const real = {
  name: function() {}
  weight: function() {}
  size: "small"
 }
 fake = td.object(real)
```

When used on a real object, td.object replaces all the functions with test-double functions but doesn't touch any property of the object that is not a function. So, fake.name is a test-double function, but td.size is still "small." The advantage here is that the real definition presumably happens somewhere in your code, and if that definition changes, the test-double object also changes, somewhat analogous to an RSpec verifying double.

In this case, the code uses the classes defined in the ES6 version of JavaScript rather than a plain JavaScript object.[9] For that, testdouble.js provides the td.constructor function. As with td.object, td.constructor has two forms: one that wraps the properties of an existing class and one that can be set free-form. The previous code sample uses the wrapper version, which takes the class object as the argument td.constructor(ProjectLoader). The return value is a stubbed constructor that you can use to instantiate objects whose methods are covered by testdouble.js:

```
const FakeLoader = td.constructor(ProjectLoader)
project.projectLoader = new FakeLoader()
```

Technically the argument to td.constructor is not a class name, but a constructor function (which means you could use td.constructor with pre-ES6 code if you were writing pre-ES6 class-like structures). If that sentence doesn't make sense to you, don't worry; the distinction isn't very important.

You can also pass an array of string function names to td.constructor, in which case you get a constructor that can create objects that respond to those function names. For the pre-ES6 crowd, it's creating an object with fake functions in its prototype object.

No matter which way you build your test-double structure, you'll eventually wind up with a function or method that is controlled by testdouble.js. Next, you need to specify the behavior of those fake functions.

Specifying Test Double Behavior

Once you have a testdouble.js function, you need to specify some behavior for it, analogous to the way RSpec's test doubles use allow and expect with chained methods like to_receive.

9. http://es6-features.org/#ClassDefinition